Experiment 11

Title: Orchestration using Docker Compose & Docker Swarm (Continuation of Experiment 6)

PART A – CONCEPT CONTINUATION (Simple Explanation)

From Experiment 6, you already know:

Tool	What it does	Limitation
`docker run`	Runs a single container	Manual, no coordination
Docker Compose	Runs multiple containers together	Single machine, no auto-healing

New Concept: Orchestration

Orchestration = Automatic management of containers

Think of it like a **restaurant manager**:

- Decides how many waiters are needed (scaling)

- Replaces a sick waiter immediately (self-healing)
- Distributes customers evenly (load balancing)

What Orchestration Adds:

Feature	What it means
Scaling	Increase/decrease number of containers
Self-healing	Restart failed containers automatically
Load balancing	Distribute traffic across containers
Multi-host	Run containers across multiple machines

The Progression Path

```
docker run  →  Docker Compose  →  Docker Swarm  →  Kubernetes
   |               |                  |                |
Single container  Multi-container    Orchestration    Advanced
                 (single host)       (basic)         orchestration
```

This experiment focuses on: *Moving from Compose → Swarm*

PART B – PRACTICAL (EXTENSION OF EXPERIMENT 6)

Prerequisites

- Docker installed (with Swarm mode enabled)
- The `docker-compose.yml` file from Experiment 6 (WordPress + MySQL)

Note: *If you don't have the file, here it is again:*

```yaml
# docker-compose.yml (from Experiment 6)
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
```

```
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Task 1: Check Current State (No Swarm)

First, make sure no containers from Experiment 6 are running:

```
# Stop any existing compose setup
docker compose down -v

# Verify no containers are running
docker ps
```

Expected: Empty list (or only unrelated containers)

Task 2: Initialize Docker Swarm

Swarm mode turns your current machine into a **manager node** of a cluster.

```
docker swarm init
```

What this command does:

- Enables Swarm mode on your Docker daemon
- Makes this node a "manager" (can control the cluster)
- Creates a join token for worker nodes (not needed for single-node)

Expected output:

```
Swarm initialized: current node (xxxxx) is now a manager.
```

Verify Swarm is active:

```
docker node ls
```

Expected output:

```
ID                            HOSTNAME    STATUS    AVAILABILITY   MANAGER STATUS
xxxxxxxxxxxx                   your-pc     Ready     Active         Leader
```

Explanation: *This shows your node is ready as a Swarm manager.*

Task 3: Deploy as a Stack (Not Just Compose)

In Swarm, we deploy a **stack** (a group of services) using the same Compose file.

```
docker stack deploy -c docker-compose.yml wpstack
```

Breaking down the command:

- `docker stack deploy` → Deploy a stack to Swarm
- `-c docker-compose.yml` → Use this Compose file
- `wpstack` → Name of the stack

What happens behind the scenes: 1. Swarm reads the Compose file 2. Creates **services** (not individual containers) 3. Services manage containers automatically

Expected output:

```
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
```

Task 4: Verify the Deployment

List all services in the stack:

```
docker service ls
```

Expected output:

```
ID             NAME                MODE         REPLICAS   IMAGE
xxxxx          wpstack_db          replicated   1/1        mysql:5.7
xxxxx          wpstack_wordpress   replicated   1/1        wordpress:latest
```

Key change: *Notice the names are* `wpstackdb` *and* `wpstack` `wordpress` *(stack name + service name)*

See detailed tasks (containers) for a service:

```
docker service ps wpstack_wordpress
```

Expected output:

```
ID             NAME                   IMAGE              NODE     DESIRED STATE   CURRE
xxxxx          wpstack_wordpress.1    wordpress:latest   your-pc   Running         Runn
```

See all running containers:

```
docker ps
```

You'll see containers with names like:

- `wpstack wordpress.1.xxxxx`
- `wpstack db.1.xxxxx`

Observation: *Containers are now* ***managed by Swarm****, not directly by you.*

Task 5: Access WordPress

Open your browser:

```
http://localhost:8080
```

You should see the WordPress setup screen (same as Experiment 6).

> ***Important:*** *Even though Swarm is managing it, the application works the same way!*

Task 6: Scale the Application (Swarm's Superpower)

This is where Swarm shines over plain Compose.

Scale WordPress from 1 to 3 replicas:

```
docker service scale wpstack_wordpress=3
```

Expected output:

```
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
```

Verify scaling:

```
docker service ls
```

Notice the REPLICAS column shows `3/3` for wordpress.

```
docker service ps wpstack_wordpress
```

You'll see **3 running instances** of WordPress.

Check containers:

```
docker ps | grep wordpress
```

You'll see **3 WordPress containers** running simultaneously!

What Just Happened?

Before Scaling	After Scaling
1 WordPress container	3 WordPress containers
No load distribution	Swarm balances traffic
Manual scaling needed	One command scaling

Important Question: How do 3 containers share port 8080?

Answer: Swarm creates an **internal load balancer**.

- All 3 containers receive traffic
- You still access `localhost:8080`
- Swarm distributes requests automatically

Task 7: Test Self-Healing (Automatic Recovery)

Self-healing = Swarm automatically replaces failed containers.

Step 1: Find a WordPress container

```
docker ps | grep wordpress
```

Copy the **CONTAINER ID** of one WordPress container.

Step 2: Kill it (simulate a crash)

```
docker kill <container-id>
```

Example:

```
docker kill a1b2c3d4e5f6
```

Step 3: Watch Swarm recreate it

```
docker service ps wpstack_wordpress
```

What you'll see:

```
ID             NAME                     IMAGE              NODE     DESIRED STATE
xxxxx          wpstack_wordpress.1      wordpress:latest   your-pc   Running
yyyyy          wpstack_wordpress.2      wordpress:latest   your-pc   Running
zzzzz          wpstack_wordpress.3      wordpress:latest   your-pc   Running
aaaaa          \_ wpstack_wordpress.3   wordpress:latest   your-pc   Shutdown
```

Notice:

- The killed container shows `Shutdown` / `Failed`
- A **new container** is automatically created
- Total replicas still = 3

Step 4: Verify new container is running

```
docker ps | grep wordpress
```

Still 3 containers running — Swarm fixed it automatically!

This is self-healing: *No manual restart needed*.

Task 8: Remove the Stack

Clean up everything:

```
docker stack rm wpstack
```

Expected output:

```
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Verify removal:

```
docker service ls
```

(Should show no services)

```
docker ps
```

(WordPress/MySQL containers should be gone)

> ***Note:*** *Volumes persist unless you delete them manually with* `docker volume prune`.

PART C – ANALYSIS (Compose vs Swarm)

Side-by-Side Comparison

Feature	Docker Compose	Docker Swarm
Scope	Single host only	Multi-node cluster
Scaling	`--scale` flag (basic, no load balancing)	`docker service scale` (built-in)
Load Balancing	No (port conflicts)	Yes (internal LB)
Self-Healing	No (must restart manually)	Yes (automatic)
Rolling Updates	No	Yes (zero downtime)
Service Discovery	Via container names	Via DNS + VIP
Use Case	Development, testing	Simple production clusters

Complexity	Low	Medium

When to Use What?

```
Development ──────► Compose
Testing     ──────► Compose
Small Production ─► Swarm
Large Production ─► Kubernetes
```

PART D – IMPORTANT OBSERVATIONS FOR STUDENTS

Observation 1: Compose File Reuse

Same YAML file works for both Compose and Swarm!

Command	Mode
`docker compose up -d`	Compose (single host, no orchestration)
`docker stack deploy`	Swarm (orchestration enabled)

Key insight: *Swarm extends Compose, not replaces it.*

Observation 2: Containers vs Services

Concept	Meaning

Container	A single running instance
Service	A definition of how to run containers (image, replicas, etc.)

In Swarm, you manage **services**, not individual containers.

Observation 3: The Port Mystery Solved

Problem: In Compose, scaling WordPress to 3 would fail because all try to use port 8080.

Solution in Swarm:

- Swarm's load balancer listens on port 8080 once
- Traffic is distributed to all 3 containers internally
- No port conflicts!

PART E – LEARNING OUTCOME CHECK

Answer These Questions (Write in Your Lab Book)

1. **Why is Compose not enough for production?**
    - (Hint: Think about scaling, failures, multiple machines)
2. **What does `docker stack deploy` do differently than `docker compose up`?**
3. **How does Swarm achieve self-healing?**
4. **What happens if you run `docker kill` on a container managed by Swarm?**
5. **Can you use the same Compose file for both development (Compose) and production (Swarm)? Why?**

PART F – OPTIONAL: Multi-Node Swarm (Advanced)

If you have access to multiple VMs or computers on the same network:

On manager node, get join token:

```
docker swarm join-token worker
```

On worker node, join the cluster:

```
docker swarm join --token <token> <manager-ip>:2377
```

Verify from manager:

```
docker node ls
```

Now your stack will distribute containers across multiple machines automatically!

Summary

You started with	You can now do
Single container (`docker run`)	Multi-container (Compose)
Manual scaling	One-command scaling (`scale`)
Manual recovery	Automatic self-healing
Single host	Multi-host cluster ready

Final Takeaway

Compose defines the application. Swarm runs it reliably.

Quick Reference Card

```
# Initialize Swarm
docker swarm init

# Deploy stack
docker stack deploy -c docker-compose.yml <stack-name>

# List services
docker service ls

# Scale service
docker service scale <stack-name_service-name>=<replicas>

# See service tasks
docker service ps <service-name>

# Remove stack
docker stack rm <stack-name>

# Leave Swarm (if needed)
docker swarm leave --force
```